Lecture 14: Memory Hierarchy I — Locality & the Memory Hierarchy

Session 17 · Mon Oct 26, 2026

Contents

Recap: Lectures 1, 10 and 12 . . . 1
Outline . . . 1
The Memory Wall . . . 2
Storage Technologies . . . 2
SRAM and DRAM . . . 3
The Memory Hierarchy . . . 3
Caching: Blocks, Hits and Misses . . . 3
Temporal Locality . . . 4
Spatial Locality . . . 4
Stride-k Reference Patterns . . . 4
Run It: Stride . . . 5
Row-Major vs. Column-Major . . . 5
Run It: Row vs. Column Order . . . 5
Run It: Time per Load by Working Set Size . . . 5
Cache-Friendly Code . . . 6
AI Systems Connection . . . 6
Why Memory Prices Rose: AI and HBM . . . 6
The Lecture in One Slide . . . 7
Practice . . . 7
Answers to Practice . . . 7
Reading . . . 8

Recap: Lectures 1, 10 and 12

Lecture 1. Summing a 4096×4096 array of `int` row by row took 5 ms. Summing the same array column by column took 132 ms, 26 times longer, with the same instructions. The reason was the cache, and we did not explain it then.

Lectures 10 and 12. A page that is not in main memory is read in from the disk by a page fault. The kernel also keeps copies of recently read file blocks in memory, in its **page cache**, so reading the same file again does not touch the disk.

Today. The CPU caches, main memory and the page cache all do the same thing: they keep a copy of data in a smaller, faster memory, close to where it is used. This lecture explains why that works and how to write code that benefits from it.

Outline

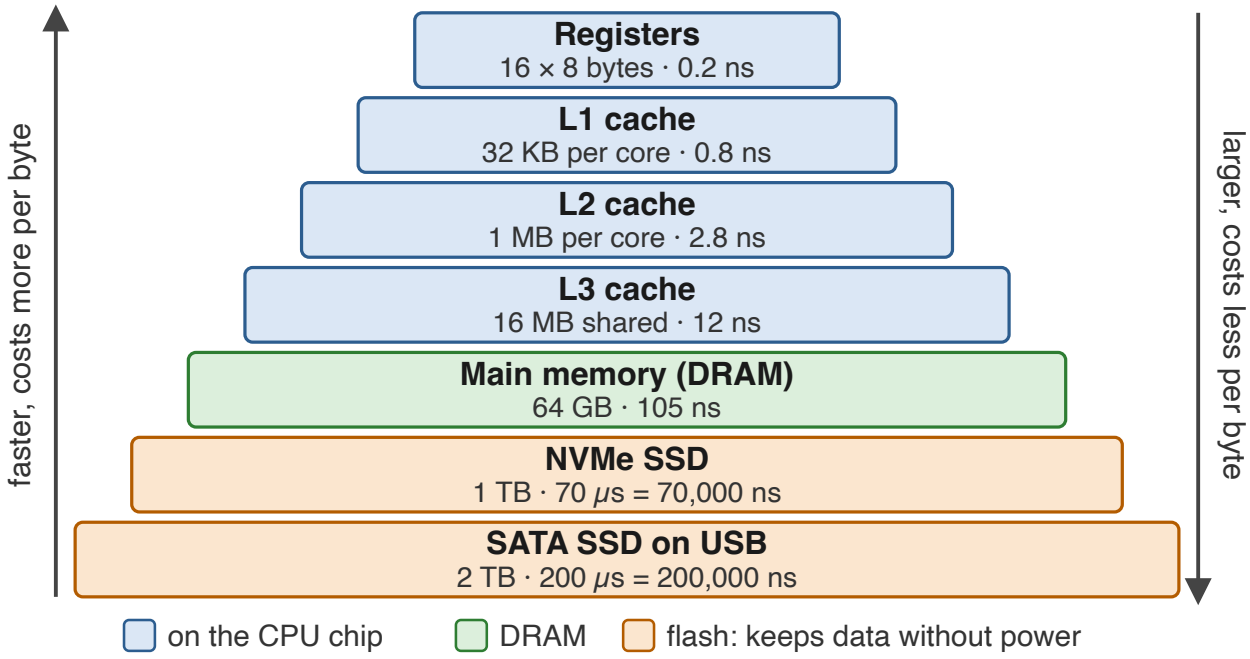
What you will learn today:

1. Why the processor spends much of its time waiting for memory
2. The four storage technologies, SRAM, DRAM, flash and disk, and how they compare
3. How a memory hierarchy works: each level keeps copies of blocks from the level below it

4. What a hit and a miss are, and who moves the blocks at each level
5. Temporal and spatial locality, and how to judge the locality of a loop
6. Three measurements on a real laptop: stride, row versus column order, and working-set size
7. Rules for writing code that uses the caches well
8. Why generating text with a large language model is limited by memory, not by arithmetic, and why that has made memory expensive

The Memory Wall

A processor works in steps timed by its clock. One tick of the clock is a **cycle**. At 5 GHz the clock ticks 5 billion times a second, so one cycle is 0.2 ns. An addition takes one cycle. Reading one value takes about 4 cycles from L1, 500 from main memory, and a million from the USB SSD:



For decades, processor speed grew much faster than memory speed. This gap is called the **memory wall**. All times were measured on a laptop with an AMD Ryzen 7 8845HS.

Storage Technologies

	SRAM	DRAM	Flash (SSD)	Hard disk
used for	CPU caches	main memory	files	files, large archives
time for one read	about 1 ns	about 100 ns	about 100 µs	about 8 ms
typical size	a few MB	8 to 64 GB	0.5 to 4 TB	4 to 30 TB
keeps data without power	no	no	yes	yes

Each column to the right is slower but holds more bytes for the same price. Most of a hard disk's 8 ms is mechanical: the arm moves to the right track, and the platter turns until the data passes under the head. The bytes right after the first one arrive almost at once, so the kernel always reads and writes whole blocks, never single bytes.

SRAM and DRAM

RAM (random-access memory) takes about the same time to read any address. The CPU caches, 0.8 to 12 ns in the pyramid, are **SRAM**, static RAM: a bit stays put as long as the power is on. Main memory, 105 ns, is **DRAM**, dynamic RAM: each bit is a charge that leaks away, so the chip must rewrite every row about every 64 ms. This is called **refresh**.

	SRAM	DRAM
one bit is	6 transistors in a loop	1 capacitor and 1 transistor
where	on the CPU chip	on separate chips, over the memory bus
a read	the value comes out at once	the charge is sensed, which drains it, so it is written back

SRAM needs 6 transistors per bit, so a chip holds only megabytes. DRAM needs 1 transistor and 1 capacitor, so it is cheap enough for gigabytes. L1 beats L3 although both are SRAM: L1 is small and next to its core, while L3 is 16 MB and shared by 8 cores.

Registers are not RAM. They have no addresses: an instruction names them directly, such as `%rax`, and each core has only 16 general-purpose registers, built into the core itself.

The Memory Hierarchy

Each level is a **cache** for the level below it: a smaller, faster memory that holds copies of some of the data stored in the larger, slower level.

Level	Holds copies of	Who moves the data	Size on the Ryzen 7 8845HS
registers	values from L1	the compiler	16 × 8 bytes
L1 cache	64-byte blocks from L2	the CPU hardware	32 KB per core
L2 cache	64-byte blocks from L3	the CPU hardware	1 MB per core
L3 cache	64-byte blocks from main memory	the CPU hardware	16 MB, shared by 8 cores
main memory	4 KB pages and file blocks from the disk	the kernel (page faults, page cache)	64 GB
SSD or disk	web pages and files from other machines	programs, such as a web browser's cache	1 TB NVMe, 2 TB SATA

It works because most programs reuse a small part of their data many times. While that part is in a fast level, the program runs at that level's speed.

Caching: Blocks, Hits and Misses

Data moves between two levels in fixed-size **blocks**: 64 bytes between main memory and the CPU caches, and 4 KB pages between the disk and main memory. Suppose a cache holds four blocks, and a program reads from block 14 and then from block 12:

```
cache holds:   block 8   block 9   block 14   block 3

read from block 14  ->  hit:  the cache returns the data. No slow access.
read from block 12  ->  miss: the cache fetches all of block 12 from the
                        level below, stores it, and returns the data. To
                        make room it evicts (throws away) a block, here 9.

cache holds:   block 8   block 12  block 14   block 3
```

A **hit** is a read whose block is already in the cache. A **miss** is a read whose block is not, so the cache must fetch the block from the level below. A miss costs the time of the slower level. The first read of any block is always a miss.

After a miss, the whole block is in the cache. Later reads of the same bytes, or of other bytes in the same block, are hits until the block is evicted. A program runs fast when most of its reads are hits.

Temporal Locality

A program has good **locality** when it uses data and instructions that are close to the ones it used recently. There are two kinds.

Temporal locality means the program uses the same memory location again soon after the first use. A location used many times costs one miss and then many hits.

```c
long sum_array(const int *a, int n) {
    long sum = 0;
    for (int i = 0; i < n; i++)
        sum += a[i];        // sum and i are used in every iteration
    return sum;
}
```

`sum` and `i` are read and written in every iteration. The compiler keeps them in registers, the fastest level. The loop's instructions also have temporal locality: the CPU fetches the same few instructions in every iteration, and they stay in the instruction cache.

Spatial Locality

Spatial locality means the program uses a memory location and then, soon after, a location near it. One miss brings in a whole 64-byte block, so the reads of its other bytes are hits.

In `sum_array`, the loop reads `a[0]`, `a[1]`, `a[2]`, … in order. They are 4 bytes apart, so one 64-byte block holds 16 of them:

```
block 0:  a[0]  a[1]  a[2] ... a[15]     a[0]: miss, a[1] to a[15]: 15 hits
block 1:  a[16] a[17] a[18] ... a[31]    a[16]: miss, then 15 hits
```

The loop misses once every 16 reads, a miss rate of 1/16.

An array has good spatial locality because its elements are next to each other in memory. A linked list does not. Each node is wherever `malloc` put it, so following `p->next` usually reads a different block, and each block brings in bytes the loop never uses.

Stride-k Reference Patterns

A loop that reads every k-th element of an array has a **stride-k** reference pattern. `sum_array` has stride 1. The larger the stride, the fewer elements the loop uses from each block it brings in:

Stride (in `ints`)	Bytes apart	`ints` used per 64-byte block	Misses per read
1	4	16	1/16
2	8	8	1/8
4	16	4	1/4
8	32	2	1/2
16 or more	64 or more	1	1

`01-stride.c` adds up all 32 Mi `ints` of a 128 MB array with stride s. It makes s passes: the first reads `a[0]`, `a[s]`, `a[2s]`, …, the next starts at `a[1]`, and so on. Every stride does the same additions on the same bytes. Only the order changes.

Run It: Stride

```
$ ./01-stride               # Linux, AMD Ryzen 7 8845HS, 64-byte blocks
stride  bytes apart  ns per int
     1            4        0.21    about 1 cycle: 15 of 16 reads hit
     2            8        0.22
     4           16        0.45
     8           32        0.61    each block still holds 2 used ints
    16           64        1.21    every read needs a new block: 6x
    32          128        3.93    skips every other block: 3x again
    64          256        4.41
```

ns per int is the run's total time divided by the 32 Mi `int`s it reads: the average cost of one `int`, not the time of one read. At stride 1 it is 0.21 ns, about one cycle (0.2 ns), so the loop runs as fast as the additions. At stride 16 every read misses, and one read from main memory takes 105 ns. The cost is still only 1.21 ns because the **prefetcher**, a circuit that sees a steady stride and fetches the next blocks early, keeps many blocks on the way at once. At stride 32 it helps less.

Row-Major vs. Column-Major

C stores a two-dimensional array **row by row**: all of row 0, then all of row 1, and so on. This is called **row-major order**. For `int a[3][4]`:

```
address:  +0      +4      +8      +12     +16     +20    ...  +44
element:  a[0][0] a[0][1] a[0][2] a[0][3] a[1][0] a[1][1] ... a[2][3]
```

The element `a[i][j]` of an array with `C` columns is at byte offset `(i * C + j) * sizeof(int)` from the start.

```
for (int i = 0; i < 4096; i++)          // int a[4096][4096], row by row:
    for (int j = 0; j < 4096; j++)      //   j changes fastest, stride 1
        sum += a[i][j];
for (int j = 0; j < 4096; j++)          // column by column: i changes
    for (int i = 0; i < 4096; i++)      //   fastest, stride 16 KB
        sum += a[i][j];
```

The row loop misses once every 16 reads. The column loop uses 4 bytes of each block, and the block is usually evicted before the next column needs it, so every read misses.

Run It: Row vs. Column Order

`02-rowcol.c` sums the same 64 MB array both ways, three times:

```
$ ./02-rowcol                  # Linux, AMD Ryzen 7 8845HS
run 1: row-major    3.4 ms, column-major  125.2 ms, 36.6x
run 2: row-major    3.4 ms, column-major  127.6 ms, 37.9x
run 3: row-major    3.4 ms, column-major  125.1 ms, 37.1x
```

Both functions do 16 Mi additions and return the same sum. The column loop is about 37 times slower. Across nine runs on this laptop the ratio was 37 to 43.

Lecture 1 showed 26 times on the same model of processor, with a different program and compiler version. The exact ratio depends on the machine and the program. The cause is the same everywhere: the column loop uses one `int` from each block it brings in.

Run It: Time per Load by Working Set Size

A program's **working set** is the data it uses over and over. `03-workingset.c` follows a ring of pointers through its working set in random order, so each load must wait for the one before it, and the prefetcher cannot guess the next address.

```
$ ./03-workingset              # Linux, AMD Ryzen 7 8845HS
working set  ns per load
     32 KB          0.8         fits in L1 (32 KB): about 4 cycles
     64 KB          2.8         fits in L2: about 14 cycles
    512 KB          3.5
      1 MB          8.2         L2 is 1 MB, but it also holds other data
      2 MB         11.4         fits in L3: about 60 cycles
      8 MB         12.5
     16 MB         29.3         L3 is 16 MB, shared with other programs
     32 MB         70.9
    256 MB        106.3         main memory (DRAM): about 530 cycles
```

Here **ns per load** is the full time of one read, because no two loads overlap. The loop and the number of loads never change, yet the time grows 130 times, from 0.8 ns to 106 ns, only because the data no longer fits in a faster level.

Cache-Friendly Code

1. **Keep data that is used together next to each other in memory.** Prefer an array of values to a linked list of nodes.
2. **Make the inner loop read memory with stride 1.** For a C array, the last index must change fastest.
3. **Keep the working set small.** Finish all the work on one piece of the data while it is in the cache, then move to the next piece. Processing a large matrix one small square tile at a time, called **blocking**, is the standard example.
4. **Keep structs small, and put fields that are used together next to each other**, so one block holds more of what the loop needs.
5. **Use local variables for values used in every iteration**, so the compiler can keep them in registers.

Most of a program's reads happen in its inner loops, so look there first. The compiler kept both loops of `02-rowcol.c` in the order they were written. Choosing a good order is the programmer's job.

AI Systems Connection

Arithmetic intensity is the number of arithmetic operations a program does for each byte it reads from memory. It tells you which part of the machine limits the program.

Generating text with a large language model produces one token (about one word) at a time. For each token, the GPU reads every weight of the model once and does about two operations with it, a multiply and an add. A model with 7 billion 16-bit weights is 14 GB:

- operations per token: about 14 billion; bytes read per token: 14 billion. The intensity is about 1 operation per byte.
- A GPU that reads memory at 500 GB/s needs 28 ms to read 14 GB, so it cannot produce more than about 35 tokens per second for one user. Its arithmetic units sit idle most of the time.

The standard fixes move fewer bytes per token. **Quantization** stores each weight in 4 bits instead of 16, so each token reads 3.5 GB. **Batching** serves many users at once: each weight is read once and used for every user's next token. That is temporal locality: one slow read, then many uses.

Why Memory Prices Rose: AI and HBM

The last slide showed that generating text is limited by how fast the GPU reads memory. So AI chips use **HBM** (high bandwidth memory): DRAM chips stacked 8 or 12 high, joined by thousands of vertical wires, and placed next to the GPU in the same package.

- One NVIDIA GB300 NVL72 rack holds 72 GPUs with 20 TB of HBM in total. That is the DRAM of more than 300 laptops like the one in this lecture.
- HBM is made from the same silicon wafers as ordinary DRAM, but for the same number of bytes it uses about 3 times as much wafer area as DDR5.

- TrendForce estimates that in 2027 HBM will use nearly 30% of all DRAM wafers but produce only about 13% of the bytes.
- Cloud companies sign multi-year contracts for server memory, and PCs and phones get what is left. Contract prices for ordinary DRAM rose 45 to 50% in the last quarter of 2025, then another 93 to 98% in the first quarter of 2026.

Sources: NVIDIA's GB300 NVL72 specifications; Micron at Hot Chips 2026; TrendForce press releases, 2026.

The Lecture in One Slide

- Reading main memory takes hundreds of cycles; reading L1 takes about 4. SSDs and disks are thousands to millions of times slower still.
- A memory hierarchy keeps copies of blocks from each slow level in a smaller, faster level. A hit is served by the fast level; a miss fetches the whole block from the level below.
- Temporal locality: the same data is used again soon. Spatial locality: nearby data is used soon. Both turn one miss into many hits.
- C stores arrays row by row. A loop whose inner index walks along a row has stride 1 and misses once per block. Walking down a column misses on every read.
- When the working set stops fitting in a level, every load pays the next level's time.

Practice

1. A loop reads an array of `double` (8 bytes each) with stride 1. Blocks are 64 bytes. What fraction of the reads miss? What if the stride is 4?
2. `int a[3][5]` starts at address `0x1000`. What is the address of `a[2][1]`?
3. `struct point { int x, y, z; }` is 12 bytes, and `struct point p[1000]` is 12,000 bytes. Version A sets all three fields of `p[i]` in one loop. Version B uses three loops: one sets every `p[i].x`, the next every `p[i].y`, the last every `p[i].z`. What is the stride of each loop, in `int`s? Which version has better locality, and does it matter much when the whole array fits in a 32 KB L1 cache?
4. Is binary search on a sorted array of 100 million `int`s cache-friendly? Think about the first few probes and the last few.
5. On the Ryzen 7 8845HS, a program's working set grows from 8 MB to 32 MB. Using the measurements in this lecture, about how much longer does each random load take, and why?
6. A hard disk spins at 7200 revolutions per minute and seeks in 4 ms on average. After the seek, it waits on average half a turn for the data to come under the head. About how long does it take to read one 4 KB block at a random place? About how long to read the next 4 KB block right after it? Why do file systems try to store a file's blocks next to each other?

Answers to Practice

1. One in 8, since a block holds 8 `double`s. With stride 4 the reads are 32 bytes apart, so 2 reads per block: one in 2 misses.
2. Offset `(2 * 5 + 1) * 4 = 44` bytes, so `0x1000 + 44 = 0x102C`.
3. Version A has stride 1: it writes all 3000 `int`s in address order. Version B makes three passes with stride 3. A has better spatial locality. Here the whole 12 KB array fits in L1, so after B's first pass the other two passes hit, and the difference is small. On an array larger than the cache, B would miss on every block three times.
4. Mostly not. The first probes are millions of elements apart, so each one is a miss and uses 4 bytes of its block. Only the last few probes fall in the same block. The probes near the middle of the array are the same for every search, so they stay in the cache across many searches: that is temporal locality.
5. From about 12.5 ns to about 71 ns, almost six times longer. 32 MB does not fit in the 16 MB L3 cache, so most loads now go to main memory.
6. One turn takes 60 / 7200 s = 8.3 ms, so half a turn is 4.2 ms. With the 4 ms seek, the read takes about 8 ms. The next block passes under the head right after, so it costs only the transfer time, microseconds. Blocks stored next to each other let one seek and one rotation serve the whole file.

Reading

- CS:APP §6.1–6.3 (Storage technologies, locality, the memory hierarchy)
- What Every Programmer Should Know About Memory, Sections 3 and 6
- The sample code in `sample code/lecture14/`